ES6 Features

- ES6 stands for ECMAScript 6 and is also known as ECMAScript 2015. It is a major version of JavaScript that introduced features designed to make JavaScript code shorter, cleaner, and easier to maintain.

ES6 Features

Feature	Main Idea
let and const	Modern variable declarations
Arrow Functions	Short function syntax
Template Literals	Strings with variables and expressions
Destructuring	Extract values from arrays and objects
Default Parameters	Default function arguments
Rest and Spread	Collect and expand values
Classes	Blueprint for objects
Modules	Share and reuse code
for...of	Iterate through values
Map	Key-value collection
Set	Unique-value collection

1. let and const

ES6 introduced let and const for declaring variables. Both are block-scoped.

```
let age = 25;
age = 30;
const name = "ABC";
console.log(age);
console.log(name);
```

Output:
30 ABC

- let → the value can be reassigned.
- const → the variable cannot be reassigned.
- Both are block-scoped.

- They are generally preferred over var in modern JavaScript.

Remember: let → can change | const → cannot be reassigned

2. Arrow Functions

Arrow functions provide a shorter syntax for writing functions.

```
const add = (a, b) => a + b;

console.log(add(10, 20));
```

Output: 30

- Uses the => syntax.
- Makes function expressions shorter.
- Commonly used with arrays and callbacks.
- Arrow functions have lexical this behavior.

Remember: Arrow Function → Shorter way to write a function

3. Template Literals

Template literals make it easy to create strings containing variables and expressions.

```
let name = "ABC";
let age = 20;

console.log(`Name: ${name}, Age: ${age}`);
```

Output: Name: ABC, Age: 20

- Uses backticks (` `).
- Use ${} to insert variables or expressions.
- Reduces string concatenation using +.

Remember: Template Literal → String + variables/expressions using backticks

4. Destructuring

Destructuring allows values to be extracted from arrays or objects into variables.

```
let numbers = [10, 20];
let [a, b] = numbers;

let student = { name: "ABC", age: 20 };
let { name, age } = student;

console.log(a, b);
console.log(name, age);
```

Output:
10 20
ABC 20

- Array destructuring uses [].
- Object destructuring uses {}.
- Makes extracting values shorter and cleaner.

Remember: Destructuring → Extract values from arrays/objects easily

5. Default Parameters

Default parameters provide a default value when a function argument is not supplied.

```
function greet(name = "Guest") {
   console.log(`Hello ${name}`);
}

greet();
greet("ABC");
```

Output:
Hello Guest
Hello ABC

- The default value is used when the argument is missing or undefined.
- Makes functions easier and safer to use.

Remember: Default Parameter → Default value for a missing argument

6. Rest and Spread Operator

The ... syntax is used as Rest to collect values and Spread to expand values.

Rest Operator

```
function sum(...numbers) {
   return numbers[0] + numbers[1] + numbers[2];
}


console.log(sum(10, 20, 30));
```

Output: 60

Spread Operator

```
let nums = [10, 20, 30];
let newNums = [...nums, 40];


console.log(newNums);
```

Output: [10, 20, 30, 40]

- Rest → collects multiple values into one variable.
- Spread → expands elements of an array or object.
- Both use the ... syntax.

Remember: Rest → Collect | Spread → Expand

7. Classes

A class provides a blueprint for creating objects. ES6 introduced class syntax to make object-oriented programming more convenient.

```
class Student {
   constructor(name, age) {
      this.name = name;
      this.age = age;
   }

   display() {
      console.log(this.name, this.age);
   }
}

let student = new Student("ABC", 20);
student.display();
```

Output: ABC 20

- class defines the class.
- constructor() initializes object data.
- this refers to the current object.
- new creates an object from the class.

8. Modules

Modules allow JavaScript code to be divided into separate files and shared between them using export and import.

Export

```
export const name = "ABC";
```

Import

```
import { name } from "./student.js";

console.log(name);
```

- export makes code available to other modules.
- import uses code exported from another module.
- Modules help organize large applications into separate files.

9. for...of

The for...of loop is used to iterate over the values of an iterable such as an array.

```
let numbers = [10, 20, 30];

for (let number of numbers) {
   console.log(number);
}
```

Output:
10 20 30

- for...of gives values from the iterable.
- It is commonly used with arrays and other iterable collections.

10. Map

Map is a collection that stores data as key-value pairs.

```
let student = new Map();

student.set("name", "ABC");
student.set("age", 20);

console.log(student.get("name"));
```

Output: ABC

- set() adds or updates a key-value pair.
- get() retrieves a value using its key.
- Map can use different types of values as keys.

Remember: Map → Key-value pairs

11. Set

Set is a collection that stores unique values.

```
let numbers = new Set();


numbers.add(10);
numbers.add(20);
numbers.add(10);


console.log(numbers);
```

The duplicate value 10 is stored only once.

- add() adds a value.
- has() checks whether a value exists.
- Set does not store duplicate values.

Remember: Set → Unique values